BreakMyAPI blueprint

Oct 11, 2026 · @dev@iecse

What it is

BreakMyAPI is a web app where a developer submits a backend API they built, and an AI agent tests it for common security and reliability mistakes, then returns a report that names each problem, shows how it was confirmed, cites the rule behind it, and explains the fix.

It is built as a 4-week portfolio project to practice the four things interviewers probe for AI + backend intern roles: LangGraph agent orchestration, retrieval (RAG), applied GenAI, and production backend engineering.

Who it is for: student and early-career developers who ship side projects and need to catch the bugs that surface in demos and interviews. The first real users are your own campus developer community.

The one-line pitch: find the bugs in your API before your interviewer does.

Out of scope on purpose: no data-destroying attacks such as SQL injection or server-side request forgery, no load or stress testing, and no scanning of APIs the user cannot prove they own. These are left out for safety and scope, and each exclusion is a design decision you can defend in an interview.

Checks

Each check is three small pieces: a **rule** stored in the rulebook (retrieved by the agent), a **test writer** that turns an endpoint into concrete requests, and a **verifier** in plain code that decides pass or fail. Adding a check is then an afternoon, not a redesign. Build the first four in weeks 1 to 2, add the rest in week 3, ending near ten.

#	Check	What it catches	Build week
1	Object-level access	User B reads or edits user A's record by changing an ID	1
2	Missing auth	An endpoint works without a valid login token	1
3	Mass assignment	A user sets a field they should not, such as role=admin	2

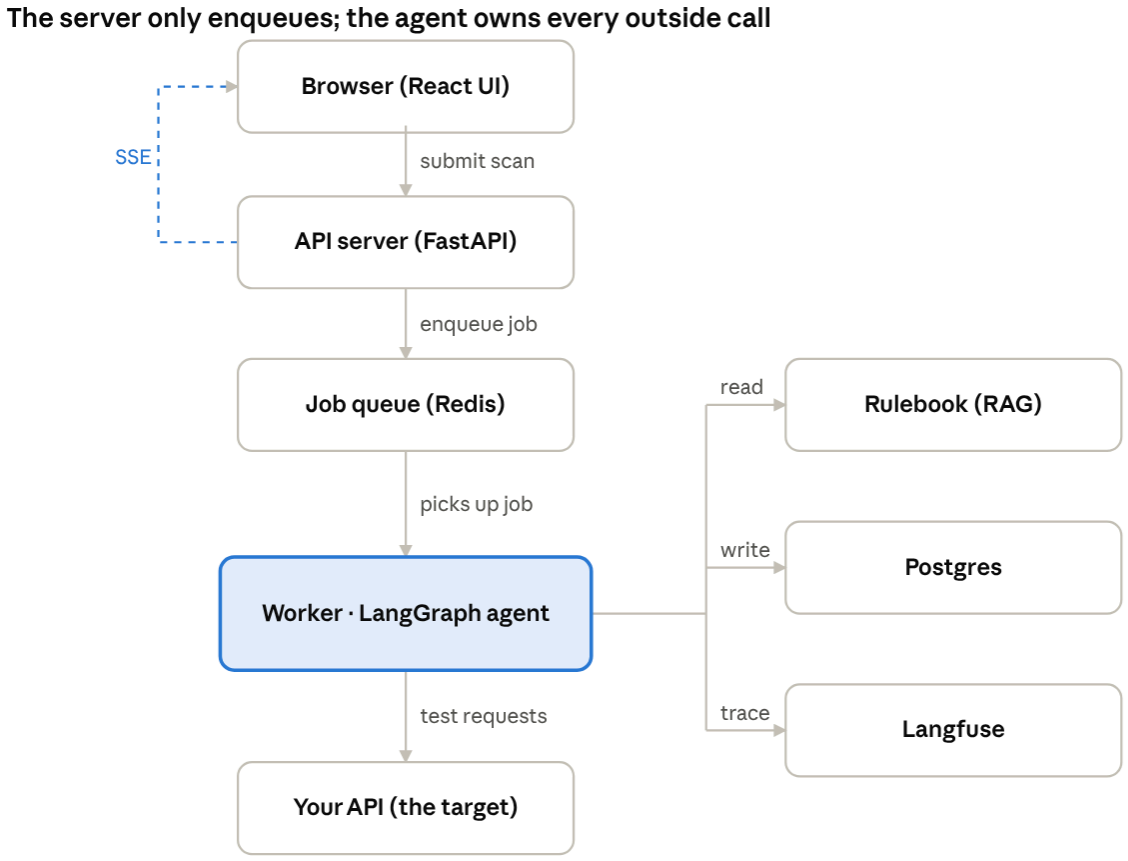
#	Check	What it catches	Build week
4	Excessive data	A response returns password hashes or fields the app never shows	2
5	Bad input handling	Wrong types or missing fields cause a 500 crash instead of a clean 400	3
6	Leaky errors	Stack traces or database errors shown to the caller	3
7	Spec mismatch	Fields or status codes differ from what the API file promises	3
8	Admin route exposure	A normal user can call an admin-only endpoint	3
9	No login rate limit	Unlimited password or OTP attempts	3
10	No page-size limit	Asking for a huge page returns the whole table	3

Checks 5 to 7 are reliability, not security. That is deliberate: almost every student API fails them, so every user gets a useful report even when their permissions are fine.

Judged, not confirmed: business-logic bugs (double-booking a seat, a negative quantity refund) are marked "needs review" rather than confirmed, because only the model can judge them.

Left out: injection and SSRF (can corrupt real data, hard to confirm safely, mature tools already cover them) and load testing (a different product, easy to misuse).

Architecture



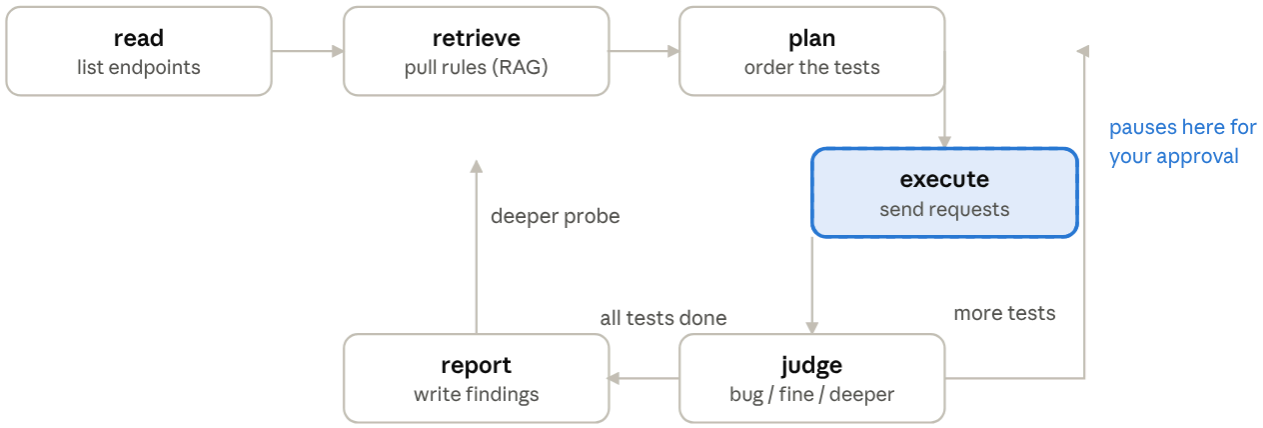
scan flow · 8 components

The API server stays thin: it validates the request, drops a job on the queue, and streams progress back over SSE. The worker runs the LangGraph agent, which is the only thing that touches the rulebook, the target API, and the trace log. Because every step writes a checkpoint to Postgres, a crashed scan resumes from the last finished step instead of starting over.

Agent graph

The agent is a LangGraph `StateGraph`. One scan is one run, and the state object carries the endpoint list, the retrieved rules, the test plan, the responses collected, and the findings so far.

The graph loops to probe deeper and pauses before risky tests



agent graph · 6 nodes, 1 loop, 1 interrupt

Nodes: `read` parses the spec, `retrieve` pulls matching rules from the vector store, `plan` orders the tests, `execute` sends requests, `judge` labels each response, `report` writes the result.

The loop: after `judge`, a conditional edge decides. If a finding looks real and deeper probing is warranted, it routes back to `plan`; when the plan is exhausted it goes to `report`. This loop is the thing interviewers mean by "agentic" rather than a fixed pipeline.

The interrupt: `execute` is compiled with an interrupt before any state-changing request (DELETE, PATCH, POST). The graph pauses, writes its checkpoint, and waits for your approval before continuing. This is LangGraph's human-in-the-loop primitive.

Durability: a Postgres checkpointer saves state after every node, keyed by scan ID, so a crash or redeploy resumes from the last finished node.

Rulebook and retrieval

Retrieval is what makes this a real RAG project and not a bolt-on: the rules you retrieve decide which tests get written, and every finding cites the rule it came from.

What goes in the rulebook: the OWASP API Security Top 10 (2023) entries, a handful of CWE descriptions, and one short card per check written by you. Each card holds the vulnerability name, which endpoint shapes it applies to, how to test it, and how to confirm it. These are chunked, embedded, and stored in pgvector.

How retrieval steers the run: for each endpoint, the agent builds a query from the endpoint's shape (does it take an ID in the path, does it require auth, does it accept a body) and retrieves the matching cards. The plan is written only from what comes back, so adding a rule card adds a behavior with no code change.

The citation: each card carries a stable source URL. When a finding is reported, the retrieved card and its link travel with it, which is what produces the "why the agent ran this test" line in the report.

The debugging story interviewers ask for: keep retrieval and generation measurable separately. If a known bug is missed, you can show whether the right card was never retrieved (a retrieval miss) or was retrieved but the test was written wrong (a generation miss). Being able to tell those apart is a named interview question.

Backend

The backend is where the "advanced backend" part of your goal lives. Keep the API server thin and push all the slow, stateful work into the worker.

Core endpoints:

Method	Path	Does
POST	/scans	Validate input, create a scan, enqueue the job, return a scan ID
GET	/scans/{id}	Return status and findings so far
GET	/scans/{id}/stream	Server-sent events: live progress as the agent works
POST	/scans/{id}/approve	Resume a paused scan after the human approves a risky test
POST	/scans/{id}/cancel	Stop a running scan

Data model (Postgres): `scans` (id, owner, target, status, cost), `endpoints` (per scan), `findings` (per scan, with severity, evidence, cited rule), and the LangGraph checkpoint tables.

The backend concepts you get to practice, and can defend:

- **Async job queue:** the POST returns immediately; a worker processes the scan. Your BullMQ experience maps straight onto this.
- **Streaming:** SSE pushes each finished step to the browser, the same pattern you used in PrepAI.
- **Idempotency:** the approve and webhook paths must tolerate being called twice without running a test twice.
- **Rate limiting, two kinds:** outbound (slow requests to the target so you never overload it) and inbound (a per-user daily scan cap so nobody runs up your model bill).

- **Secrets:** the two test tokens are encrypted at rest and never written to logs or the trace.
- **Config and health:** environment-based config, a health endpoint, and structured logs.

Safety rules

The thing that could go wrong (someone scanning an API they do not own) is also the best interview material, because handling it is the OWASP Top 10 for Agentic Applications in practice. Build these from day one and lead with them in the README.

1. **Ownership proof before any scan.** The user must place a verification file or DNS record on the target, which you check before the first request. No proof, no scan. This is the single most important rule.
2. **Approval gate on anything destructive.** The agent pauses before any DELETE, PATCH, or POST and waits for explicit approval. Read-only tests run freely.
3. **Outbound rate limit.** Requests to the target are capped (for example 2 per second) so a scan never looks like an attack or overloads a small server.
4. **Least privilege.** The scan uses only the two test-user tokens the owner supplied, never admin credentials.
5. **Treat responses as untrusted.** Data coming back from the target (error text, JSON fields) is never followed as an instruction, which is the defense against indirect prompt injection.
6. **Secret hygiene.** Tokens are encrypted at rest, redacted from logs and traces, and deleted when the scan is.
7. **Hard scope limits.** No injection payloads, no SSRF, no load testing. The allowed test types are a fixed list.

Evaluation

This is the section that turns the project from a demo into a portfolio piece. Without it, you have another agent demo; with it, you have numbers you can defend.

Test targets: run deliberately vulnerable open-source APIs locally (OWASP crAPI, VAmPI, and similar practice apps). Verify the current list and their known-bug inventories before you rely on them. These give you ground truth: you know exactly which bugs exist.

The baseline to beat: a single prompt that is handed one endpoint and asked to guess its vulnerabilities, with no retrieval, no execution, no verification. Your agent must beat it, and showing the gap is the point.

Metrics, per check type:

- **Precision:** of the bugs reported, how many are real (the opposite of crying wolf).
- **Recall:** of the known bugs, how many were found.
- **Cost per scan:** model spend, so you can talk about the quality-versus-cost tradeoff.
- **Time per scan.**

Grading: wherever possible a deterministic check decides pass or fail (did user B receive user A's record, yes or no). The LLM-as-judge is used only for the ambiguous cases, and you note that it is a measurement instrument, not ground truth.

CI gate: the eval suite runs on every push via GitHub Actions. A change that drops recall below the last known-good number fails the build. This is the "evals are the unit tests of AI engineering" idea, made concrete.

The resume line this produces: something like "found 27 of 31 known vulnerabilities across 4 practice APIs at about $0.04 per scan, versus 11 of 31 for a single-prompt baseline" (your real numbers, once measured).

Stack and the decisions that are yours

Suggested stack:

Layer	Choice	Why
Agent	Python + LangGraph	What the job posts ask for; the whole point of the project
API server	FastAPI	Async, typed, pairs naturally with the Python worker
Queue	Redis	Simple, free tier, matches your BullMQ background
Database	Postgres + pgvector	One store for state, findings, and the rulebook vectors
Tracing	Langfuse	Free tier or self-host; gives you the observability talking point
Frontend	React	You already have it from PrepAI
Model	Start on a free or cheap tier	Keep cost per scan measurable from day one

Decisions you should make yourself (and write down why):

- **Spec file or live URL as input.** Start with the OpenAPI spec: it is more deterministic and far easier to evaluate. Add live-URL crawling only if time allows.
- **One worker or many.** One is fine for a demo; mention how you would scale out.
- **Which model, and when to switch.** Route cheap models for simple judging, a stronger one only for hard cases. This is a direct interview answer.
- **How deep the loop can go.** Cap the probe depth so a scan cannot loop forever.

Keep a short decision log in the repo. Every entry is an interview answer you have already rehearsed. The rule from the hiring guide is blunt: "the agent chose it" is not an acceptable answer, so for anything the AI generates, be able to explain it back without looking.

Four-week plan

Each week ends with something that works end to end, so you are never left with a half-built system if time runs short.

Week	Build	Deliverable at the end
1	Spec parsing, the rulebook ingested into pgvector, the single-prompt baseline, and the FastAPI + queue skeleton	A scan can be submitted and the baseline runs against one practice API
2	The LangGraph graph: state, the read-retrieve-plan-execute-judge-report nodes, the execute tool, the checkpointer, and the approval interrupt. Checks 1 to 4	The agent runs a full scan on a practice API and pauses for approval
3	Checks 5 to 10, Langfuse tracing, the eval harness, and the eval gate in CI	Precision and recall numbers for all checks, running in CI
4	Ownership verification, the cited report UI, deploy, and the README with real numbers and honest failure cases	Live, shareable, and launched to your campus developer community

If you fall behind: cut checks, not the evaluation. Four working checks with real numbers beat ten checks you cannot measure. The evaluation is the part that makes the project count.

Interview map

Every part of this project answers a question from the 2026 AI-engineer interview bank. Build the part, and you have lived the answer.

Part you build	Question it prepares you for
The LangGraph loop and interrupt	When do you use an agent instead of a fixed workflow?
Ownership proof, approval gate, untrusted responses	How do you secure a tool-using agent and enforce authorization at action time?
Separating retrieval from generation in evals	A known bug was missed, even though the rule exists. How do you debug it?
The eval harness and CI gate	How do you build an eval set and decide whether a change can ship?
Deterministic checks plus the judge	Can an LLM-as-judge be trusted?
Model routing and cost per scan	Quality improved but cost became unacceptable. What do you do?
The checkpointer and resume-on-crash	How do you make a long-running agent survive failure?
The whole project	Walk me through an AI system you shipped.

The last row is the one that matters most. Interviews open with "walk me through something you built," and this project gives you a story with a user, a baseline, real numbers, honest failures, and a decision log behind every choice.

Risks and open questions

Risks:

- **Scope creep.** The pluggable-check design makes it tempting to add more checks. Stop near ten and spend the time on evaluation instead.
- **Cost from abuse.** A public tool that makes model calls can run up your bill. The per-user daily cap is not optional.
- **Explaining AI-written code.** If you cannot walk through a part, it hurts you in the interview. Hand-write the graph, one tool, and the idempotency check yourself at least once.
- **Thin eval data.** Every check needs known-buggy and known-clean examples, or the numbers mean nothing. More checks means more eval work.

Open questions for you to answer before week 1:

- Spec file or live URL as the input for v1? (Recommendation: spec file.)

- Which model provider and tier will you start on, and what is the free allowance?
- Who are the first ten real users, and when in week 4 do you hand it to them?
- Which practice APIs will you use as ground truth, and do their documented bugs still match the current versions?

Sources that shaped this plan:

- AI Engineer Interview Questions 2026 (PracHub)
- How to Get an AI Engineer Job in 2026 (Louis-François Bouchard)
- OWASP API Security Top 10 (2023)
- OWASP Top 10 for Agentic Applications (2026)
- What is LangGraph? (Atlan)